

PROGRAM-2

TITLE: Develop a multi-stage Dockerfile for container orchestration

Structure of the Program:

Program-2:

Dockerfile

Src/index.js

package.json

package-lock.json

Node_modules

Step-1: Create a Dockerfile

```
GNU nano 7.2 Dockerfile
FROM node:20-alpine AS builder
#set wd
WORKDIR /app
#copy package files and install dependencies
COPY package.json package-lock.json ./
RUN npm install
#copy application src code
COPY . .
#build the application
RUN npm run build
#stage 2:production
FROM node:20-alpine
#set working directory
WORKDIR /app
#copy only necessary files from build stage
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
RUN npm install --omit=dev
COPY --from=builder /app/dist ./dist

#expose the application port
EXPOSE 3000
#start application
CMD ["node","dist/index.js"]
```

Step-2: You can write package.json or create it using npm init-y.

```
1RV24MC025_CHANDAN_K@chandan-Vivobook-ASUSLaptop-M6500QC-M6500QC:~/prog2$ ls
Dockerfile  node_modules  package.json  package-lock.json  src  -
```

Step-3: Create a src folder and write index.js file inside it

```
GNU nano 7.2 index.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from multi-stage Docker!');
});
app.listen(PORT, () => {
  console.log('Server running on port ${PORT}');
});
```

Step-4: install node dependencies

```
1RV24MC025_CHANDAN_K@chandan-Vivobook-ASUSLaptop-M6500QC-M6500QC:~/prog2/src$ npm install

up to date, audited 69 packages in 1s

18 packages are looking for funding
  run `npm fund` for details

2 vulnerabilities (1 moderate, 1 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
1RV24MC025_CHANDAN_K@chandan-Vivobook-ASUSLaptop-M6500QC-M6500QC:~/prog2/src$
```

Step-5: Create a docker image

- Sudo docker build -t prg-2 .

```
1RV24MC025_CHANDAN_K@chandan-Vivobook-ASUSLaptop-M6500QC-M6500QC:~/prog2$ sudo docker build -t prg2 .
[+] Building 24.1s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 630B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488ac03a370d448
=> => resolve docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488ac03a370d448
=> => sha256:cb3325e64457574e24f2246e3da3376946e473d636209e1390eac47b50b26a3 1.26MB / 1.26MB
=> => sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488ac03a370d448 1.67kB / 1.67kB
=> => sha256:fcbb8f7d018707c656a4da2eea8a15f2893d2258093fea9ff2ea5ea1c8a82112 1.72kB / 1.72kB
=> => sha256:e08b0510ba947015cacdda3d23dc761e399971c074a1bf32eea7e44510524 6.52kB / 6.52kB
=> => sha256:1074353e0db2c1d81d5af2671e56e00cf5738486f5762609ea33d606f88612 3.86MB / 3.86MB
=> => sha256:8d06ba6946d1f299d8f962e37906c77006df33d47050430a57f7893ec35af697 42.78MB / 42.78MB
=> => extracting sha256:1074353e0db2c1d81d5af2671e56e00cf5738486f5762609ea33d606f88612
=> => sha256:fd1849a5c548bc65ee47a64498951bda5d40e87d08efe9dca69b5c8cdedc7a52 443B / 443B
=> => extracting sha256:8d06ba6946d1f299d8f962e37906c77006df33d47050430a57f7893ec35af697
=> => extracting sha256:cb3325e64457574e24f2246e3da3376946e473d636209e1390eac47b50b26a3
=> => extracting sha256:fd1849a5c548bc65ee47a64498951bda5d40e87d08efe9dca69b5c8cdedc7a52
=> [internal] load build context
=> => transferring context: 99.70kB
=> [builder 2/6] WORKDIR /app
=> [builder 3/6] COPY package.json package-lock.json ./
=> [builder 4/6] RUN npm install
=> [builder 5/6] COPY . .
=> [builder 6/6] RUN npm run build
=> [stage-1 3/6] COPY --from=builder /app/package.json ./
=> [stage-1 4/6] COPY --from=builder /app/package-lock.json ./
=> [stage-1 5/6] RUN npm install --omit=dev
=> [stage-1 6/6] COPY --from=builder /app/dist ./dist
=> => exporting to image
=> => exporting layers
=> => writing image sha256:6f7362f4e4779a74dbc4523caa5d039833698c93edeae9ab06483343ae16c2c
=> => naming to docker.io/library/prg2
docker:default
0.0s
0.0s
0.0s
4.5s
0.0s
0.0s
0.0s
0.0s
0.0s
8.4s
0.0s
2.1s
0.0s
0.0s
0.0s
1.4s
7.1s
0.1s
2.3s
0.9s
0.0s
0.0s
0.1s
0.1s
0.2s
0.1s
4.2s
1.2s
0.5s
0.1s
0.1s
4.2s
0.1s
0.3s
0.2s
0.0s
0.0s
```

Step-6: Run the container

- Sudo docker run -it -p 3000:3000 prg-2

```
1RV24MC025_CHANDAN_K@chandan-Vivobook-ASUSLaptop-M6500QC-M6500QC:~/prog2$ sudo docker run -it -p 3000:3000 prog2
Server running on port ${PORT}
```

Step-7: go to browser and type <http://localhost:3000>



Hello from multi-stage Docker!